

napari Plugin System — Quick Reference Notes

This guide provides a structured overview of the napari plugin architecture, focusing on discovery, manifest-driven design, and implementation.

1. Reader Selection Logic

When napari auto-discovers a plugin for a file, it follows this sequence:

1. **Extension Query:** Checks installed plugins for a handler matching the file extension (e.g., .evi).
2. **User Preference Lookup:** Checks `settings.plugins.extension2reader` to see if you have saved a preferred reader.
3. **Conflict Resolution:**
 - **Single Reader:** Auto-selects the only available handler.
 - **Multiple Readers:** Raises a `MultipleReaderError` if more than one plugin claims the extension.

Usage Tip: The default `viewer.open()` command uses `plugin='napari'` (built-in). To trigger auto-discovery, pass `plugin=None`.

Resources: [napari reader docs](#) | [npe2 source](#)

2. npe2 — Manifest-Driven Design

The modern `npe2` engine optimizes performance by scanning only the `napari.yaml` manifest at startup. Python code is loaded "lazily" only when you actually trigger a command.

Example Manifest (napari.yaml)

```
None
name: napari-evi-reader
contributions:
  commands:
    - id: napari-evi-reader.get_reader
```

```
python_name: napari_evi_reader._reader:get_reader
readers:
- command: napari-evi-reader.get_reader
  filename_patterns: ["*.evi"]
```

See the [npe2 manifest schema](#) for more details.

3. Plugin Contribution Types

Plugins declare functionality in the manifest, linking back to a specific Python command.

Contribution Type	Description
Readers	Processes files into image layers.
Writers	Saves layers back into files.
Widgets	Adds custom UI panels to the right-side dock.
Menus & Keybindings	Integrates toolbar options or keyboard shortcuts.
Themes	Customizes the visual color scheme of the interface.

4. Implementation Methods

Method	Style	Recommendation
npe2 Manifest	Uses <code>napari.yaml</code> + <code>pyproject.toml</code> .	Modern, recommended approach.
hook_implementation	Uses Python decorators.	Legacy (npe1). Slow and deprecated.

Method	Style	Recommendation
Runtime Registration	Calls <code>viewer.window.add_dock_widget()</code> in a script.	Best for quick scripts; no packaging needed.

5. Custom Manifest Integration

The `napari.yaml` file defines your plugin's capabilities. It is discovered via the entry-point defined in your `pyproject.toml`:

None

```
[project.entry-points."napari.manifest"]
napari-evi-reader = "napari_evi_reader:napari.yaml"
```

6. Distribution & Installation

Method	Command	Use Case
Editable Install	<code>pip install -e .</code>	Development (changes apply instantly).
Standard Local	<code>pip install .</code>	Standard local installation.
Wheel (.whl)	<code>python -m build</code>	Sharing files directly with colleagues.
Global (PyPI)	<code>twine upload dist/*</code>	Public release for the community.

Note: Plugins published to PyPI with the `Framework :: napari` classifier are automatically indexed on the [napari hub](#), which hosts over 600 plugins.

Further Reading:

- [Publishing to napari hub](#)

[Official Python Packaging Guide](#)

Fiji (ImageJ2) Plugin System

Comprehensive Overview for Developers and Researchers

The Fiji (ImageJ2) ecosystem represents a powerful, extensible framework designed for scientific image analysis. While "Fiji" is a distribution of ImageJ, its underlying architecture—powered by the SciJava software stack—enables a modular approach to plugin development and distribution.

Core Architecture: The SciJava Framework

At the heart of ImageJ2 is the SciJava framework. This modularity ensures that plugins are not just "add-ons" but integrated components that can interact with the core application and other plugins seamlessly.

- **Plugin Discovery:** Unlike legacy systems that require manual registration, ImageJ2 uses automatic discovery. By using the `@Plugin` annotation in your Java code, the system identifies and loads your tool at runtime.

- Decoupled Logic: The framework separates the data (Image), the display (UI), and the processing (Plugins), making the code more maintainable and reusable.
- Extensibility: Developers can extend everything from file formats (via Bio-Formats) to custom UI widgets and menu structures.

Evolution: ImageJ1 vs. ImageJ2

Understanding the distinction between the legacy system and the modern framework is crucial for effective development.

Feature	ImageJ1 (Legacy)	ImageJ2 (Modern)
Plugin Loading	Manual classpath scanning	Automatic via Scijava annotations
UI Framework	Tied to Java AWT	UI-agnostic (Swing, JavaFX, etc.)
Data Model	8-bit, 16-bit, 32-bit (Limited)	N-dimensional (ImgLib2)
Distribution	Individual .jar or .class files	Update Sites
Scripting	ImageJ Macro language	Polyglot (Python, Groovy, JS, etc.)

The Plugin Lifecycle and Development

Creating a plugin for Fiji involves a few streamlined steps, leveraging modern build tools like Maven.

1. Project Structure

Most modern Fiji plugins are managed using Maven. This handles dependencies (like ImgLib2 or Scijava-common) automatically, ensuring your plugin works across different Fiji installations.

2. The @Plugin Annotation

The magic of discovery happens here. A simple annotation tells Fiji where your plugin belongs in the menu:

- **Menu Path:** Define exactly where your tool appears (e.g., `Plugins > Analytics > My Tool`).
- **Priority:** Control the order in which plugins are loaded or executed.

3. Parameters and Inputs

ImageJ2 utilizes "Parameter Harvesting." Instead of writing complex dialog code, you define inputs using the `@Parameter` annotation, and the framework automatically generates the appropriate user interface (text boxes, sliders, file pickers).

Distribution via Update Sites

Fiji revolutionized plugin sharing through **Update Sites**. This mechanism allows developers to host their tools on a remote server, which users can subscribe to directly within the Fiji "Help > Update" menu.

- **Seamless Updates:** When you release a new version, users are notified automatically.
- **Dependency Management:** Update sites can depend on other sites, ensuring all necessary libraries are installed.
- **Personal Sites:** Every developer can host their own site on the ImageJ wiki or via GitHub Pages.

Essential Reference Links

To dive deeper into the technical specifications and community tutorials, please refer to the following official resources:

- **Official ImageJ Wiki:** <https://imagej.net/>
- **Developer Guide:** <https://imagej.net/develop/>
- **Scijava Documentation:** <https://scijava.org/>
- **GitHub Repository:** <https://github.com/imagej/imagej2>
- **Fiji Update Sites List:** <https://imagej.net/update-sites/>

Prepared by:  Person

Last Updated:  Date